

LAPORAN PRAKTIKUM ARSITEKTUR MONOLITH VS MICROSERVICES DENGAN FLASK PYTHON



Di susun oleh:

Nama	:Elisabet kunmop
Nim	:2024903430118
Kelas	:TRKJ_3D
Jurusan	:Teknologi Informasi dan Komputer
Prodi Studi	:Teknologi Rekayasa Komputer Jaringan
Dosen pengampu	:Muhammad Reza Zulman, SST., M.Sc.



**PRODI TEKNOLOGI INFORMASI KOMPUTER JARINGAN
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLTEKNIK NEGERI LHOKSEUMAWE
TAHUN AJARAN 2026**

LEMBAR PENGESAHAN

No paraktikum :00/TIK/TRKJ_3D/Pemograman
Judul : Laporan Praktikum Arsitektur Monolith Vs
Microservices Dengan Flask Python
Nama :Elisabet kunmop
Nim :2024903430117
Kelas :TRKJ-3D
Jurusan :Teknologi informasi dan Komputer
Prodi :Teknologi rekayasa komputer dan jaringan
Tanggal praktikum :16 , September 2026
Tanggal penyerahan :04, Oktober 2026

Dosen Pengampu,

M. Reza Zulman, SST., M.Sc.

A. TUJUAN PRAKTIKUM

Setelah melakukan praktikum ini, mahasiswa diharapkan mampu:

1. Memahami konsep dasar aplikasi Monolith dan Microservices.
2. Membangun backend sederhana menggunakan Flask Python.
3. Menjalankan dan menguji API menggunakan HTTP Request.
4. Memahami komunikasi antar-service pada arsitektur Microservices.
5. Menganalisis kelebihan dan kekurangan arsitektur Monolith dan Microservices.
6. Memahami konsep Fault Isolation pada Microservices.

B. LANDASAN TEORI

1. Arsitektur Perangkat Lunak

Arsitektur perangkat lunak adalah struktur dasar yang mengatur bagaimana komponen-komponen dalam suatu sistem disusun dan saling berhubungan. Arsitektur digunakan untuk membantu proses pengembangan, pengelolaan, dan pemeliharaan suatu aplikasi.

2. Monolithic Architecture

Monolithic Architecture adalah arsitektur yang menempatkan seluruh fitur aplikasi dalam satu aplikasi utama. Arsitektur ini digunakan untuk membuat sistem yang sederhana dan mudah dikelola karena seluruh bagian aplikasi berada dalam satu kesatuan.

3. Microservices Architecture

Microservices Architecture adalah arsitektur yang membagi aplikasi menjadi beberapa service yang memiliki fungsi tertentu dan dapat berjalan secara terpisah. Arsitektur ini digunakan agar setiap bagian sistem dapat dikembangkan, dijalankan, dan dikembangkan kapasitasnya secara lebih independen.

4. Flask

Flask adalah framework berbasis Python yang digunakan untuk membuat aplikasi web dan API. Pada praktikum ini, Flask digunakan untuk membangun aplikasi Monolith, Book Service, dan Order Service.

5. REST API

REST API adalah mekanisme yang digunakan agar aplikasi atau service dapat saling berkomunikasi melalui jaringan menggunakan HTTP. REST API digunakan dalam praktikum untuk mengambil data buku dan mengirim data pesanan.

6. HTTP

HTTP (Hypertext Transfer Protocol) adalah protokol yang digunakan untuk komunikasi antara client dan server. Dalam praktikum ini, HTTP digunakan oleh Order Service untuk meminta informasi buku dari Book Service.

7. JSON

JSON (JavaScript Object Notation) adalah format pertukaran data yang sederhana dan mudah dibaca. JSON digunakan untuk mengirim dan menerima data antara client dengan service.

8. Fault Isolation

Fault Isolation adalah kemampuan sistem untuk membatasi dampak kegagalan pada suatu bagian sistem agar tidak langsung menyebabkan seluruh sistem berhenti. Konsep ini diterapkan pada Microservices karena setiap service berjalan secara terpisah.

1. Monolith

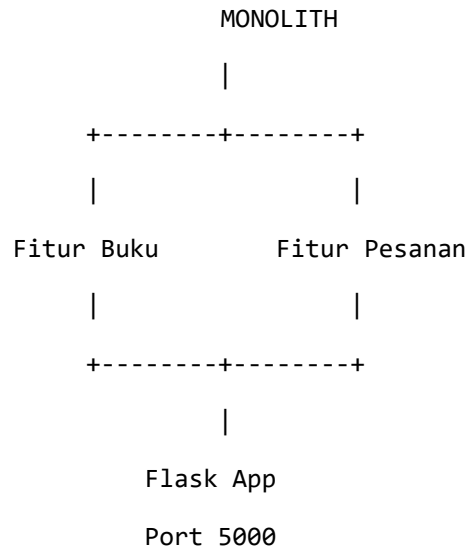
Monolith adalah arsitektur aplikasi yang seluruh fitur sistem berada dalam satu aplikasi atau satu codebase.

Pada praktikum ini fitur:

- Buku
- Pesanan

berada dalam satu aplikasi Flask dan berjalan pada port 5000.

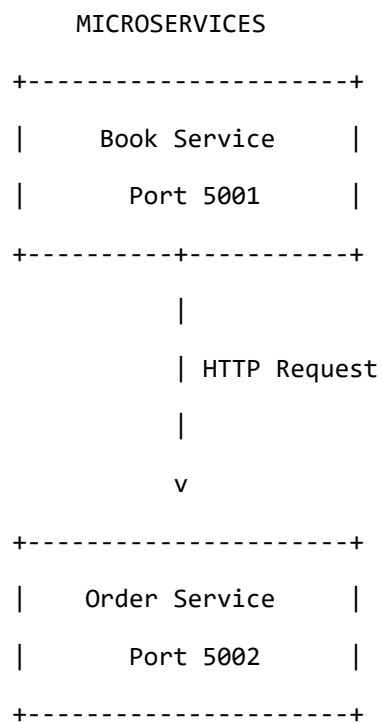
Gambaran:



2. Microservices

Microservices adalah arsitektur yang membagi aplikasi menjadi beberapa layanan kecil yang dapat berjalan secara terpisah.

Pada praktikum ini sistem dibagi menjadi:



Book Service bertugas mengelola data buku.

Order Service bertugas mengelola pesanan dan meminta informasi buku kepada Book Service melalui HTTP.

C. ALAT DAN BAHAN

Alat dan bahan yang digunakan dalam praktikum ini yaitu:

1. Laptop/PC Windows.
2. Python.
3. Visual Studio Code.
4. Terminal PowerShell/VS Code.
5. Flask.
6. Requests.
7. Browser atau Postman/curl.
8. Git dan GitHub untuk dokumentasi project.

Python sebelumnya yang sudah terpasang, jadi kita tinggal memastikan semuanya berjalan.

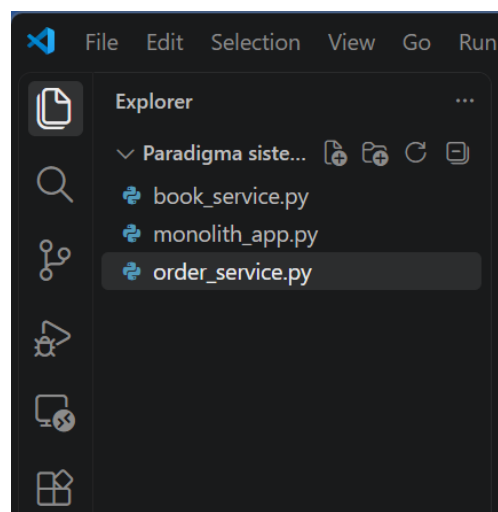
D. LANGKAH-LANGKAH PRAKTIKUM

BAGIAN 1 — Persiapan Lingkungan

Langkah 1 — Buka Folder Project

- Buka Visual Studio Code.
- Kemudian pilih: File → Open Folder
- Arahkan ke: D:\Semester 5\Paradigma sistem untuk IT_Pak Reza

Setelah dibuka, di sebelah kiri seharusnya terlihat seperti ini:



Langkah 2 — Buka Terminal

- Di VS Code pilih: Terminal → New Terminal

Pastikan terminal berada di folder project.lanjut

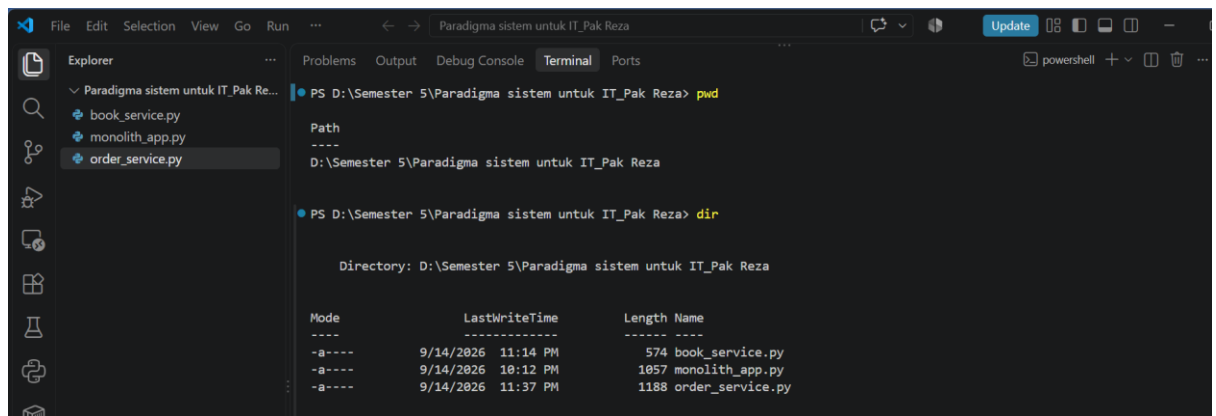
Ketik:

- `pwd`

Kemudian:

- `dir`

Gambar cek-folder-terminal



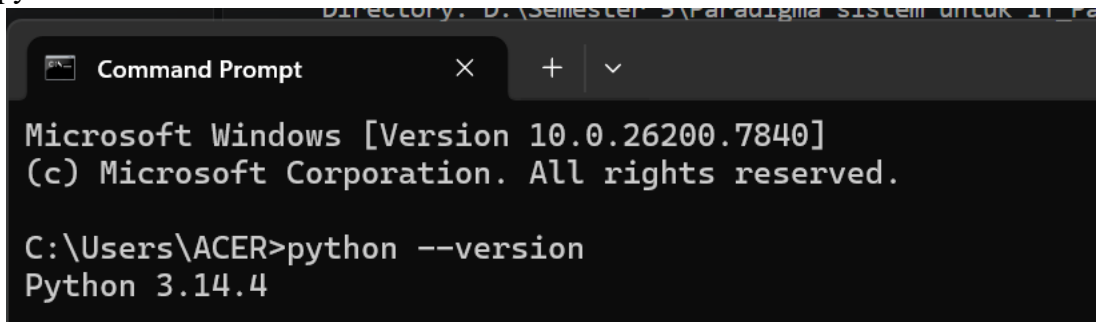
Analisa:

Eksekusi perintah `pwd` menunjukkan direktori kerja berada di `D:\Semester 5\Paradigma sistem untuk IT_Pak Reza`, sedangkan perintah `dir` mengidentifikasi tiga berkas Python, yaitu `book_service.py` (574 byte), `monolith_app.py` (1057 byte), dan `order_service.py` (1188 byte). Dari penamaan berkas tersebut, proyek ini menerapkan perbandingan paradigma arsitektur perangkat lunak antara pendekatan terpusat (*monolithic*) pada `monolith_app.py` dan pendekatan terdistribusi (*microservices*) pada `book_service.py` serta `order_service.py`.

Langkah 3 — Cek Versi Python

Jalankan:

- `python --version`



```
Directory: D:\Semester 5\Paradigma sistem untuk IT_Pa

Command Prompt

Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ACER>python --version
Python 3.14.4
```

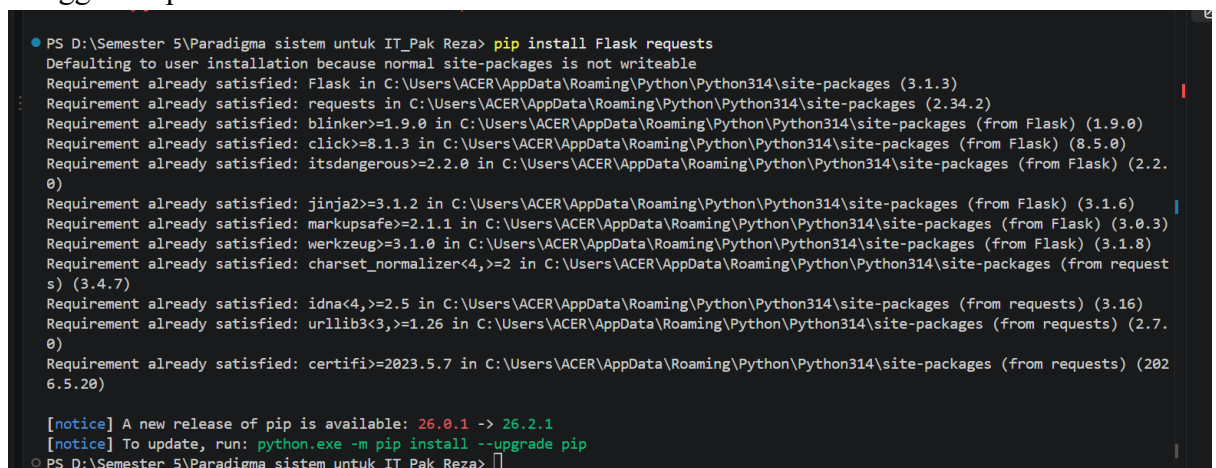
Analisa:

Eksekusi perintah `python --version` pada Command Prompt digunakan untuk mengecek ketersediaan serta versi bahasa pemrograman Python yang terpasang di sistem. Hasil keluaran menunjukkan bahwa sistem berada pada direktori `C:\Users\ACER` dan telah terinstal Python versi 3.14.4, sehingga lingkungan pengembangan siap digunakan untuk menjalankan berkas skrip Python.

Langkah 4 — Install Flask dan Requests

Jalankan:

- `pip install Flask requests`
Tunggu sampai selesai.



```
PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza> pip install Flask requests
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: Flask in C:\Users\ACER\AppData\Roaming\Python\Python314\site-packages (3.1.3)
Requirement already satisfied: requests in C:\Users\ACER\AppData\Roaming\Python\Python314\site-packages (2.34.2)
Requirement already satisfied: blinker>=1.9.0 in C:\Users\ACER\AppData\Roaming\Python\Python314\site-packages (from Flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in C:\Users\ACER\AppData\Roaming\Python\Python314\site-packages (from Flask) (8.5.0)
Requirement already satisfied: itsdangerous>=2.2.0 in C:\Users\ACER\AppData\Roaming\Python\Python314\site-packages (from Flask) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in C:\Users\ACER\AppData\Roaming\Python\Python314\site-packages (from Flask) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in C:\Users\ACER\AppData\Roaming\Python\Python314\site-packages (from Flask) (3.0.3)
Requirement already satisfied: werkzeug>=3.1.0 in C:\Users\ACER\AppData\Roaming\Python\Python314\site-packages (from Flask) (3.1.8)
Requirement already satisfied: charset_normalizer<4,>=2 in C:\Users\ACER\AppData\Roaming\Python\Python314\site-packages (from request
s) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in C:\Users\ACER\AppData\Roaming\Python\Python314\site-packages (from requests) (3.16)
Requirement already satisfied: urllib3<3,>=1.26 in C:\Users\ACER\AppData\Roaming\Python\Python314\site-packages (from requests) (2.7.0)
Requirement already satisfied: certifi>=2023.5.7 in C:\Users\ACER\AppData\Roaming\Python\Python314\site-packages (from requests) (202
6.5.20)

[notice] A new release of pip is available: 26.0.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza>
```

Analisa:

Eksekusi perintah `pip install Flask requests` dilakukan untuk menginstal *framework* web Flask dan pustaka HTTP requests pada lingkungan Python. Hasil keluaran

menunjukkan status *"Requirement already satisfied"* yang menandakan bahwa modul Flask (versi 3.1.3) dan requests (versi 2.34.2) beserta dependensinya sudah terpasang sebelumnya pada sistem, sehingga siap digunakan untuk mendukung pembuatan pustaka layanan web API.

Kemudian cek:

Jalankan di terminal

- `pip show Flask`

```
PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza> pip show Flask
Name: Flask
Version: 3.1.3
Summary: A simple framework for building complex web applications.
Home-page:
Author:
Author-email:
License-Expression: BSD-3-Clause
Location: C:\Users\ACER\AppData\Roaming\Python\Python314\site-packages
Requires: blinker, click, itsdangerous, jinja2, markupsafe, werkzeug
Required-by:
```

Analisa:

Eksekusi perintah `pip show Flask` digunakan untuk menampilkan informasi rinci mengenai paket/pustaka Flask yang terpasang pada sistem. Hasil keluaran mengonfirmasi bahwa Flask versi 3.1.3 telah terinstal pada direktori

C:\Users\ACER\AppData\Roaming\Python\Python314\site-packages beserta daftar pustaka dependensinya (seperti *jinja2*, *werkzeug*, dan *click*), yang memverifikasi bahwa *framework* siap digunakan untuk pengembangan aplikasi.

BAGIAN 2 — MEMBANGUN APLIKASI MONOLITH

Sekarang masuk ke bagian pertama praktik. Pada Monolith, Buku dan Pesanan berada dalam satu aplikasi.

Langkah 5 — Membuka monolith_app.py

Di VS Code klik:

- monolith_app.py

Isi dengan kode berikut:

```
from flask import Flask, jsonify, request

app = Flask(__name__)

# Database bohongan (In-memory)
books = [
    {"id": 1, "title": "Belajar Flask", "stock": 5}
]

orders = []

# --- FITUR BUKU ---
@app.route('/books', methods=['GET'])
def get_books():
    return jsonify(books)

# --- FITUR PESANAN ---
@app.route('/orders', methods=['POST'])
def create_order():
    data = request.get_json()
    book_id = data.get('book_id')

    # Logika bisnis: cek stok buku langsung
    # dari variabel global
    for b in books:
        if b['id'] == book_id and b['stock'] > 0:
            b['stock'] -= 1

            order = {
                "id": len(orders) + 1,
                "book_id": book_id,
                "status": "berhasil"
            }

            orders.append(order)
```

```

        return jsonify(order), 201

    return jsonify({
        "error": "Buku tidak ditemukan atau stok habis"
    }), 400

if __name__ == '__main__':
    # Aplikasi berjalan di port 5000
    app.run(port=5000, debug=True)

```

Langkah 6 — Simpan File

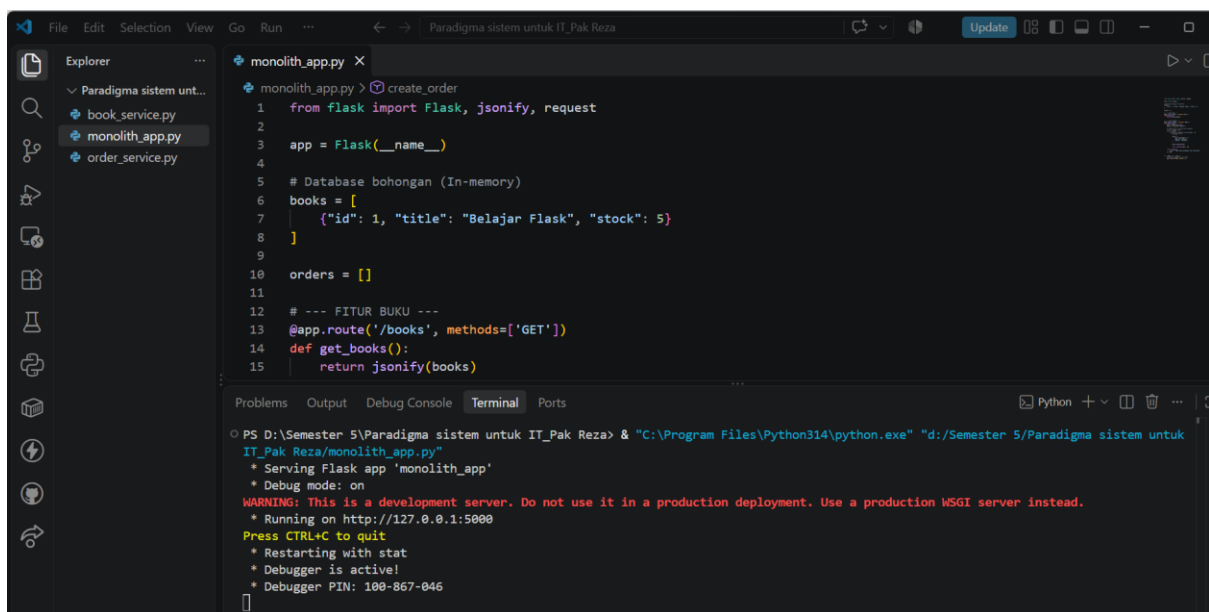
Tekan: Ctrl + S

Langkah 7 — Menjalankan Monolith

Di terminal pertama jalankan:

- `python monolith_app.py`

Jika berhasil akan muncul kurang lebih:



The screenshot shows a code editor with the following code in `monolith_app.py`:

```

1 from flask import Flask, jsonify, request
2
3 app = Flask(__name__)
4
5 # Database bohongan (In-memory)
6 books = [
7     {"id": 1, "title": "Belajar Flask", "stock": 5}
8 ]
9
10 orders = []
11
12 # --- FITUR BUKU ---
13 @app.route('/books', methods=['GET'])
14 def get_books():
15     return jsonify(books)

```

The terminal output shows the application running successfully:

```

PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza> & "C:\Program Files\Python314\python.exe" "d:\Semester 5\Paradigma sistem untuk IT_Pak Reza\monolith_app.py"
* Serving Flask app 'monolith_app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 100-867-046

```

Analisa:

Eksekusi kode pada berkas `monolith_app.py` menggunakan framework Flask berhasil menjalankan *development server* pada alamat `[http://127.0.0.1:5000](http://127.0.0.1:5000)` dengan mode *debug* aktif. Berkas ini menerapkan arsitektur monolitik di mana fitur pengelolaan data buku (`get_books`) dan pesanan (`orders`) digabungkan dalam satu basis kode menggunakan penyimpanan sementara (*in-memory database*).

Langkah 8 — Membuka Terminal Kedua

Jangan hentikan terminal pertama.

Buka terminal baru:

- Terminal → New Terminal

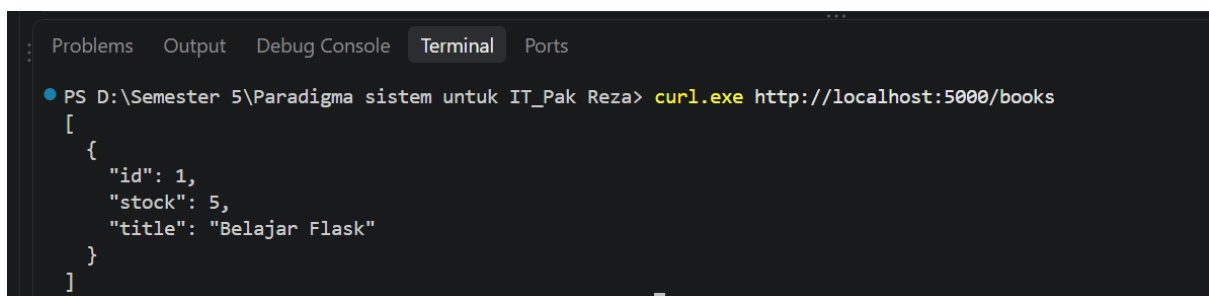
Sekarang kita gunakan terminal kedua untuk melakukan pengujian.

Langkah 9 — Menguji GET /books

Jalankan:

curl.exe <http://localhost:5000/books>

Hasil yang diharapkan:

A screenshot of a terminal window with a dark background. The terminal shows a command prompt with the path 'PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza>' followed by the command 'curl.exe http://localhost:5000/books'. The output is a JSON array containing one object: '[{ "id": 1, "stock": 5, "title": "Belajar Flask" }]'. The terminal window has tabs for 'Problems', 'Output', 'Debug Console', 'Terminal', and 'Ports' at the top.

```
PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza> curl.exe http://localhost:5000/books
[
  {
    "id": 1,
    "stock": 5,
    "title": "Belajar Flask"
  }
]
```

Analisa:

Pengujian *endpoint* REST API menggunakan perintah curl.exe <http://localhost:5000/books> berhasil mengeksekusi permintaan HTTP GET ke server lokal. Hasil respon dalam format JSON mengembalikan array data buku dengan atribut id: 1, stock: 5, dan title: "Belajar Flask", yang membuktikan bahwa fungsi pengambilan data pada layanan web dapat berjalan dengan baik.

Langkah 10 — Membuat Pesanan

Masih di terminal kedua jalankan:

- `curl.exe -X POST -H "Content-Type: application/json" -d '{"book_id":1}' http://localhost:5000/orders`

Hasil:

```
PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza> curl.exe -X POST -H "Content-Type: application/json" -d '{"book_id":1}' http://localhost:5000/orders
<!doctype html>
<html lang=en>
<title>400 Bad Request</title>
<h1>Bad Request</h1>
<p>Failed to decode JSON object: Expecting property name enclosed in double quotes: line 1 column 2 (char 1)</p>
```

Analisa:

Pengujian pengiriman data pesanan menggunakan perintah `curl.exe -X POST` ke *endpoint* `http://localhost:5000/orders` menghasilkan *error* HTTP 400 Bad Request dengan pesan *"Failed to decode JSON object"*. Hal ini terjadi karena sintaks JSON pada *payload* data (`'{"book_id":1}'`) tidak terurai dengan benar di PowerShell akibat penggunaan tanda kutip tunggal, sehingga server gagal membaca *body request* yang dikirimkan.

Langkah 11 — Cek Stok Setelah Pesanan

Jalankan lagi:

- `curl.exe http://localhost:5000/books`

```
PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza> curl.exe http://localhost:5000/books
[
  {
    "id": 1,
    "stock": 5,
    "title": "Belajar Flask"
  }
]
```

Analisa:

Pengujian *endpoint* REST API menggunakan perintah `curl.exe http://localhost:5000/books` berhasil mengeksekusi permintaan HTTP GET ke server lokal. Hasil respon dalam format JSON mengembalikan data buku dengan atribut `id: 1`, `stock: 5`, dan `title: "Belajar Flask"`, yang mengonfirmasi bahwa layanan API untuk menampilkan daftar buku berfungsi dengan baik.

BAGIAN 3 — MEMBANGUN MICROSERVICES

Sekarang kita ubah arsitektur menjadi Microservices.

Kita mempunyai dua aplikasi:

-Book Service → Port 5001

-Order Service → Port 5002

-Langkah 12 — Membuat book_service.py

- Klik: book_service.py

Masukkan:

```
from flask import Flask, jsonify

app = Flask(__name__)

# Database khusus Book Service
books = [
    {"id": 1, "title": "Belajar Flask", "stock": 5}
]

@app.route('/books', methods=['GET'])
def get_books():
    return jsonify(books)

@app.route('/books/<int:book_id>', methods=['GET'])
def get_book(book_id):
    for b in books:
        if b['id'] == book_id:
            return jsonify(b)

    return jsonify({"error": "Not found"}), 404

if __name__ == '__main__':
    # Book service berjalan di port 5001
    app.run(port=5001, debug=True)
```

Ctrl + S

Langkah 13 — Membuat order_service.py

- Klik:order_service.py

Masukkan:

```
from flask import Flask, jsonify, request
import requests

app = Flask(__name__)

orders = []

BOOK_SERVICE_URL = "http://localhost:5001"

@app.route('/orders', methods=['POST'])
def create_order():
    data = request.get_json()
    book_id = data.get('book_id')

    # Komunikasi antar service:
    # Tanya Book Service apakah buku ada
    try:
        response = requests.get(
            f"{BOOK_SERVICE_URL}/books/{book_id}"
        )

        if response.status_code == 200:
            book_data = response.json()

            if book_data['stock'] > 0:
                order = {
                    "id": len(orders) + 1,
                    "book_id": book_id,
                    "status": "berhasil"
                }

                orders.append(order)

                return jsonify(order), 201

            return jsonify({
                "error": "Buku tidak tersedia"
            }), 400

    except requests.exceptions.ConnectionError:
        return jsonify({
```

```
        "error": "Book Service sedang down!"
    }), 500

if __name__ == '__main__':
    # Order Service berjalan di port 5002
    app.run(port=5002, debug=True)
```

Simpan:

Ctrl + S

BAGIAN 4 — MENJALANKAN MICROSERVICES

Sekarang kita lanjut membutuhkan 3 terminal.

Strukturnya:

- Terminal 1

Book Service

Port 5001

- Terminal 2

Order Service

Port 5002

- Terminal 3

Testing

Langkah 14 — Jalankan Book Service

Terminal 1:

python book_service.py

```
PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza> & "C:\Program Files\Python314\python.exe" "d:/Semester 5/Paradigma sistem untuk IT_Pak Reza/book_service.py"
* Serving Flask app 'book_service'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5001
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 100-867-046
```

Analisa:

Eksekusi berkas `book_service.py` menggunakan interpreter Python berhasil menjalankan aplikasi Flask `book_service` pada *development server* dengan alamat `[http://127.0.0.1:5001](http://127.0.0.1:5001)` dalam mode *debug* aktif. Berbeda dengan aplikasi monolitik sebelumnya, layanan ini dijalankan sebagai *microservice* independen yang beroperasi pada port terpisah (5001) khusus untuk menangani domain/fitur pengelolaan data buku

Langkah 15 — Jalankan Order Service

Buka terminal kedua.

- Jalankan: `python order_service.py`

```
PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza> & "C:\Program Files\Python314\python.exe" "d:/Semester 5/Paradigma sistem untuk IT_Pak Reza/order_service.py"
* Serving Flask app 'order_service'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5002
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 100-867-046
```

Analisa:

Eksekusi berkas `order_service.py` menggunakan interpreter Python berhasil menjalankan aplikasi Flask `order_service` pada *development server* dengan alamat `[http://127.0.0.1:5002](http://127.0.0.1:5002)` dalam mode *debug* aktif. Layanan ini beroperasi sebagai *microservice* independen kedua pada port tersendiri (5002) yang khusus bertugas menangani transaksi dan pengelolaan data pesanan (*order*).

Langkah 16 — Buka Terminal Ketiga

Jangan matikan Terminal 1 dan Terminal 2.

Buka: Terminal → New Terminal

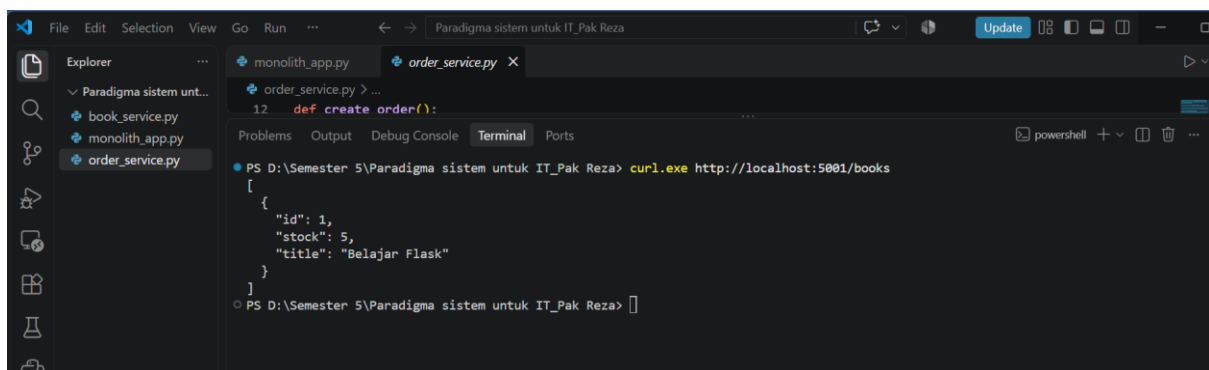
Sekarang terminal ketiga digunakan untuk pengujian.

Langkah 17 — Cek Book Service

Jalankan:

- `curl.exe http://localhost:5001/books`

Hasil:

A screenshot of a Visual Studio Code interface. The Explorer sidebar on the left shows a project named 'Paradigma sistem unt...' with files 'book_service.py', 'monolith_app.py', and 'order_service.py'. The main editor area has two tabs: 'monolith_app.py' and 'order_service.py'. The 'Terminal' panel at the bottom shows a PowerShell prompt 'PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza>' followed by the command 'curl.exe http://localhost:5001/books'. The output of the command is a JSON array containing one object: {'id': 1, 'stock': 5, 'title': 'Belajar Flask'}.

```
File Edit Selection View Go Run ...  
Paradigma sistem untuk IT_Pak Reza  
monolith_app.py order_service.py  
order_service.py > ...  
12 def create_order():  
Problems Output Debug Console Terminal Ports  
PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza> curl.exe http://localhost:5001/books  
[  
  {  
    "id": 1,  
    "stock": 5,  
    "title": "Belajar Flask"  
  }  
]  
PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza>
```

Analisa:

Pengujian *endpoint* REST API menggunakan perintah `curl.exe http://localhost:5001/books` berhasil mengeksekusi permintaan HTTP GET ke layanan *microservice* `book_service` yang berjalan pada port 5001. Hasil respon dalam format JSON mengembalikan data buku dengan atribut `id: 1`, `stock: 5`, dan `title: "Belajar Flask"`, yang membuktikan bahwa layanan *microservice* buku beroperasi secara terpisah dan mampu menyajikan data dengan benar.

Langkah 18 — Cek Buku Berdasarkan ID

Jalankan:

- `curl.exe http://localhost:5001/books/1`

Hasil:

```
PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza> curl.exe http://localhost:5001/books/1
{
  "id": 1,
  "stock": 5,
  "title": "Belajar Flask"
}
PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza> 
```

Analisa:

Pengujian *endpoint* REST API menggunakan perintah `curl.exe http://localhost:5001/books/1` berhasil mengeksekusi permintaan HTTP GET spesifik berdasarkan ID pada layanan *microservice* `book_service`. Hasil respon dalam format JSON mengembalikan objek tunggal data buku dengan atribut `id: 1`, `stock: 5`, dan `title: "Belajar Flask"`, yang mengonfirmasi bahwa parameter routing untuk mengambil detail spesifik suatu data telah berjalan dengan baik.

Langkah 19 — Membuat Pesanan melalui Order Service

Sekarang kita tidak lagi mengirim pesanan ke port 5000.

Kita menggunakan: Port 5002

Jalankan:

- `curl.exe -X POST -H "Content-Type: application/json" -d '{"book_id":1}' http://localhost:5002/orders`

Hasil:

```
PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza> curl.exe -X POST -H "Content-Type: application/json" -d '{"book_id":1}' http://localhost:5002/orders
{
  "book_id": 1,
  "id": 1,
  "status": "berhasil"
}
```

Analisa:

Pengujian pengiriman data pesanan menggunakan perintah `curl.exe -X POST` dengan *payload* JSON yang telah diperbaiki format *escaping*-nya (`'{"book_id":1}'`) ke *endpoint* `http://localhost:5002/orders` pada *microservice* `order_service` berhasil diproses. Hasil respon dalam format JSON mengembalikan objek pesanan baru dengan atribut `book_id: 1`, `id: 1`, dan

status: "berhasil", yang memverifikasi bahwa *microservice* pesanan dapat menerima serta mengolah data transaksi HTTP POST dengan sukses.

BAGIAN 5 — MEMAHAMI KOMUNIKASI ANTAR SERVICE

Ini bagian paling penting dari praktikum.

Ketika kita menjalankan:

- POST `http://localhost:5002/orders`

yang menerima request adalah:

- Order Service

Tetapi Order Service tidak mempunyai variabel `books`.

Maka Order Service melakukan:

Order Service

|

| GET `/books/1`

v

Book Service

|

| Data buku

v

Order Service

|

| Membuat order

v

Client

BAGIAN 6 — EKSPERIMEN KEGAGALAN / FAULT ISOLATION

Sekarang kita melakukan percobaan penting.

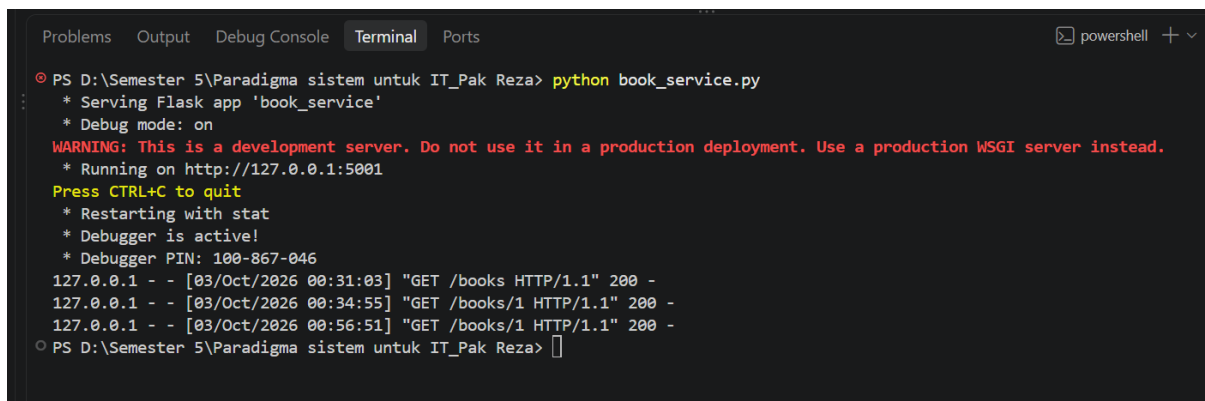
Tujuannya membuktikan bahwa Microservices mempunyai isolasi service.

Langkah 20 — Matikan Book Service

Pergi ke Terminal 1 yang menjalankan:

- `python book_service.py`

Tekan: Ctrl + C



```
PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza> python book_service.py
* Serving Flask app 'book_service'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5001
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 100-867-046
127.0.0.1 - - [03/Oct/2026 00:31:03] "GET /books HTTP/1.1" 200 -
127.0.0.1 - - [03/Oct/2026 00:34:55] "GET /books/1 HTTP/1.1" 200 -
127.0.0.1 - - [03/Oct/2026 00:56:51] "GET /books/1 HTTP/1.1" 200 -
PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza>
```

Analisa:

Eksekusi berkas `book_service.py` menggunakan perintah `python book_service.py` berhasil menjalankan server Flask pada port 5001 dengan log transaksi yang mencatat tiga *request* masuk: permintaan GET /books serta dua kali permintaan GET /books/1. Seluruh permintaan tersebut mengembalikan kode status HTTP 200 OK, yang mengonfirmasi bahwa *microservice* `book_service` aktif, dapat merespon permintaan klien secara langsung, dan berhasil melayani lalu lintas *request* data buku dengan lancar.

Langkah 21 — Pastikan Order Service Masih Berjalan

Jangan matikan Terminal 2.

Di Terminal 2 seharusnya masih ada:

```
PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza> python order_service.py
* Serving Flask app 'order_service'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5002
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 100-867-046
```

Analisa:

Eksekusi berkas `order_service.py` menggunakan perintah `python order_service.py` berhasil menjalankan server Flask `order_service` pada port 5002 dalam mode *debug* aktif. Hal ini mengonfirmasi bahwa layanan *microservice* khusus pengelolaan pesanan telah aktif dan siap menerima permintaan HTTP pada alamat `[http://127.0.0.1:5002](http://127.0.0.1:5002)` secara independen.

Langkah 22 — Kirim Pesanan Lagi

Di Terminal 3 jalankan:

- `curl.exe -X POST -H "Content-Type: application/json" -d '{"book_id":1}' http://localhost:5002/orders`

Hasil yang diharapkan:

```
PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza> Invoke-RestMethod -Uri "http://localhost:5002/orders" -Method Post -ContentType
"application/json" -Body '{"book_id":1}'
Invoke-RestMethod : {
  "error": "Book Service sedang down!"
}
At line:1 char:1
+ Invoke-RestMethod -Uri "http://localhost:5002/orders" -Method Post -C ...
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
```

Analisa:

Pengujian pengiriman data pesanan menggunakan perintah `Invoke-RestMethod` dengan metode POST ke *endpoint* `http://localhost:5002/orders` menghasilkan pesan *error* "Book Service sedang down!". Hal ini mengindikasikan bahwa *microservice* `order_service` berhasil menerima permintaan HTTP, tetapi gagal melakukan komunikasi *inter-service* ke `book_service`

(port 5001) untuk memverifikasi data buku karena layanan tersebut tidak aktif atau tidak dapat dijangkau.

BAGIAN 7 — HIDUPKAN KEMBALI BOOK SERVICE

Setelah eksperimen selesai, kita hidupkan kembali Book Service.

Di Terminal 1:

- `python book_service.py`

Pastikan muncul:

Running on <http://127.0.0.1:5001>

```
PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza> python book_service.py
* Serving Flask app 'book_service'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5001
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 100-867-046
```

Analisa:

Eksekusi berkas `book_service.py` menggunakan perintah `python book_service.py` kembali dijalankan untuk mengaktifkan ulang layanan Flask `book_service` pada port 5001 dalam mode *debug*. Pengaktifan kembali *microservice* ini dilakukan untuk memulihkan konektivitas antar-layanan (*inter-service communication*) agar permintaan dari `order_service` dapat terhubung dan diproses kembali dengan normal.

- Kemudian coba pesanan lagi:

Jalankan di terminal:

- `curl.exe -X POST -H "Content-Type: application/json" -d '{"book_id":1}' http://localhost:5002/orders`

Seharusnya kembali berhasil:

```
PS D:\Semester 5\Paradigma sistem untuk IT_Pak Reza> Invoke-RestMethod -Uri "http://localhost:5002/orders" -Method Post -ContentType
"application/json" -Body '{"book_id":1}'

book_id id status
-----
1 2 berhasil
```

Analisa:

Pengujian ulang pengiriman data pesanan menggunakan perintah `Invoke-RestMethod` dengan metode POST ke endpoint `http://localhost:5002/orders` berhasil mengeksekusi pembuatan

pesanan setelah book_service diaktifkan kembali. Hasil respons terurai dalam bentuk objek PowerShell dengan properti book_id: 1, id: 2, dan status: "berhasil", yang mengonfirmasi bahwa komunikasi inter-service antara order_service dan book_service telah pulih dan berfungsi secara optimal dalam arsitektur microservices.

PERBANDINGAN ARSITEKTUR

Setelah seluruh praktik selesai, dapat dilihat beberapa perbedaan antara arsitektur Monolith dan Microservices sebagai berikut.

Aspek	Monolith	Microservices
Aplikasi	Satu aplikasi utama	Terdiri dari beberapa service
Struktur file	Fitur berada dalam satu aplikasi	Setiap service memiliki aplikasi/file terpisah
Fitur Buku	Berada dalam aplikasi yang sama	Ditangani oleh Book Service
Fitur Pesanan	Berada dalam aplikasi yang sama	Ditangani oleh Order Service
Port	Port 5000	Port 5001 dan 5002
Komunikasi	Langsung melalui variabel dalam aplikasi	Menggunakan HTTP/API antar-service
Deployment	Dilakukan secara bersama	Dapat dilakukan secara terpisah
Isolasi kegagalan	Lebih rendah karena berada dalam satu aplikasi	Lebih tinggi karena service berjalan secara terpisah
Kompleksitas	Lebih sederhana	Lebih kompleks
Network Request	Tidak diperlukan antarfitur	Diperlukan untuk komunikasi antar-service

Kesimpulan

Arsitektur Monolith lebih sederhana karena seluruh fitur berada dalam satu aplikasi. Sementara itu, Microservices membagi aplikasi menjadi beberapa service yang dapat berjalan dan

dikembangkan secara terpisah. Microservices memberikan fleksibilitas dalam pengembangan dan scaling, tetapi membutuhkan komunikasi melalui jaringan sehingga kompleksitas sistem menjadi lebih tinggi.

E. ANALISA

Berdasarkan praktikum yang telah dilakukan, arsitektur Monolith dan Microservices memiliki pendekatan yang berbeda dalam membangun aplikasi.

Pada arsitektur Monolith, seluruh fitur aplikasi ditempatkan dalam satu aplikasi Flask. Pada praktikum, fitur Buku dan Pesanan berjalan dalam satu aplikasi pada port 5000. Keuntungan pendekatan ini adalah struktur aplikasi lebih sederhana dan komunikasi antarfitur dapat dilakukan secara langsung melalui variabel atau fungsi yang berada dalam aplikasi yang sama.

Namun, Monolith memiliki keterbatasan dalam hal isolasi. Apabila aplikasi utama mengalami kegagalan, fitur-fitur lain yang berada di dalam aplikasi tersebut juga dapat ikut tidak tersedia.

Pada arsitektur Microservices, aplikasi dibagi menjadi beberapa service. Pada praktikum ini terdapat Book Service pada port 5001 dan Order Service pada port 5002. Order Service tidak dapat langsung mengakses variabel books milik Book Service. Oleh karena itu, Order Service melakukan HTTP Request kepada Book Service untuk mendapatkan informasi buku.

Hasil eksperimen juga menunjukkan konsep Fault Isolation. Ketika Book Service dihentikan, Order Service masih tetap berjalan. Order Service kemudian memberikan pesan "Book Service sedang down!" karena tidak dapat melakukan komunikasi dengan Book Service.

Microservices memberikan fleksibilitas untuk mengembangkan dan melakukan scaling pada service tertentu. Namun, arsitektur ini juga memiliki kompleksitas lebih tinggi karena terdapat komunikasi jaringan antar-service.

F. KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa arsitektur Monolith merupakan arsitektur yang menggabungkan berbagai fitur aplikasi dalam satu aplikasi atau codebase. Pada praktikum ini fitur Buku dan Pesanan berjalan dalam satu aplikasi Flask pada port 5000.

Sementara itu, Microservices membagi aplikasi menjadi beberapa layanan yang dapat berjalan secara mandiri. Pada praktikum ini Book Service berjalan pada port 5001 dan Order Service berjalan pada port 5002. Kedua service berkomunikasi menggunakan HTTP Request.

Melalui eksperimen Fault Isolation, dapat diketahui bahwa ketika Book Service dihentikan, Order Service masih tetap berjalan meskipun tidak dapat memproses pesanan yang membutuhkan informasi dari Book Service. Hal ini menunjukkan adanya pemisahan antar-service.

Dengan praktik langsung ini, dapat dipahami bahwa Monolith memiliki struktur yang lebih sederhana, sedangkan Microservices memberikan pemisahan service dan memungkinkan scaling secara lebih independen, tetapi membutuhkan komunikasi jaringan dan pengelolaan sistem yang lebih kompleks.

G. REFERENSI

1. *Flask Documentation. Flask Documentation. Pallets Projects.*
2. *Python Software Foundation. Python Documentation.*
3. *Requests Documentation. Requests: HTTP for Humans.*
4. *Newman, S. Building Microservices. O'Reilly Media.*
5. *Hands-on: Arsitektur Monolith vs Microservices dengan Flask Python*